Diseño de Sistemas de información



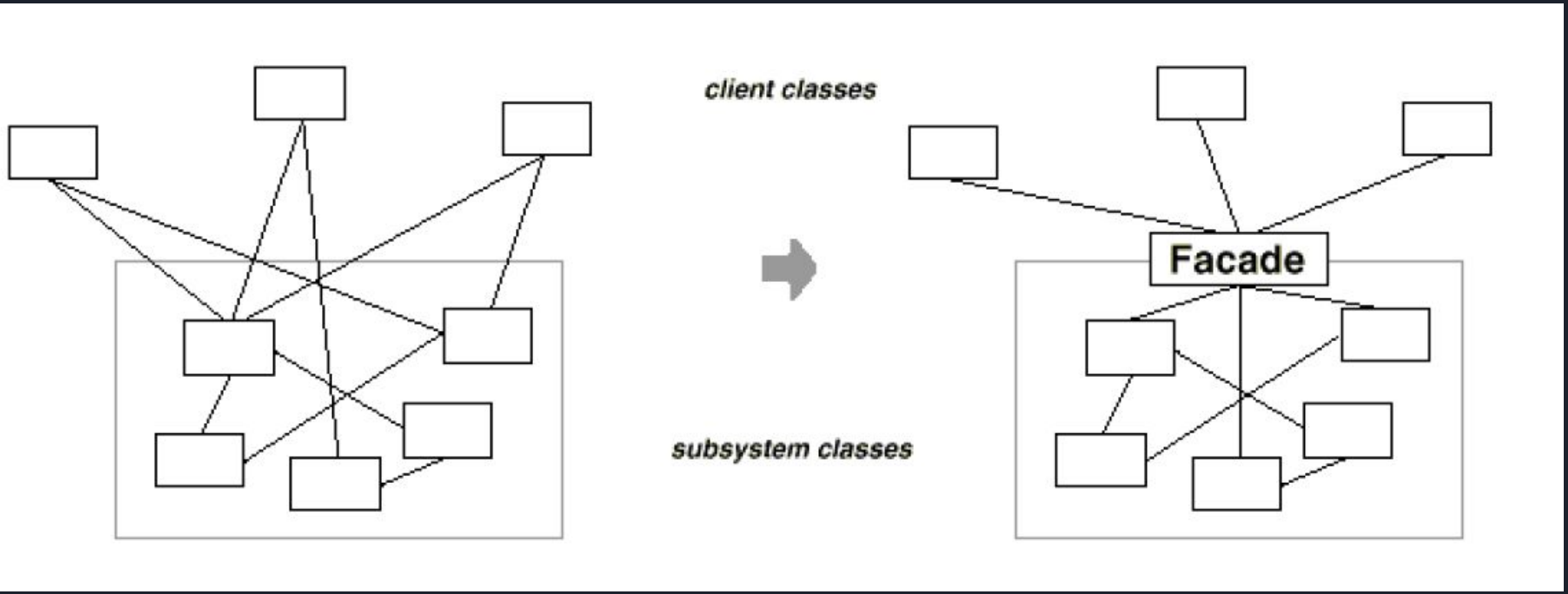
Catálogo de patrones

		Propósito		
		de creación	de estructura	de comportamiento
Alcance	A nivel de clase	·Factory Method	·Adapter	·Interpreter ·Template Method
	A nivel de objeto	·Abstract Factory ·Builder ·Prototype ·Singleton	·Adapter ·Bridge ·Composite ·Decorator ·Facade ·Proxy	·Chain of Responsibility ·Command ·Iterator ·Mediator ·Memento ·Flyweight ·Observer ·State ·Strategy ·Visitor

FACADE (FACHADA)

- NOMBRE DEL PATRÓN: Facade
- CLASIFICACIÓN DEL PATRÓN: de estructura
- INTENCIÓN: Proporcionar una interfaz unificada para un conjunto de interfaces en un subsistema.
- MOTIVACIÓN:
  - Estructurar un sistema en subsistemas reduce la complejidad.
  - En el diseño queremos minimizar la comunicación y las dependencias entre los subsistemas.
  - Una forma es introducir un objeto fachada que proporcione una única interfaz simplificada a las funcionalidades del subsistema.

FACADE



FACADE (FACHADA)

- PARTICIPANTES
  - Fachada
    - Sabe qué clases del subsistema son responsables ante una petición.
    - Delega las peticiones de los clientes en los objetos apropiados del subsistema.
  - Clases del subsistema
    - Implementan la funcionalidad
    - Realizan las operaciones expuestas por el Facade
    - No conocen al Facade

FACADE (FACHADA)

- APLICABILIDAD
  - Proporcionar una interfaz única para un subsistema complejo.
  - Desacoplar el subsistema de sus clientes y de otros subsistemas.
  - Dividir en capas nuestros subsistemas.

FACADE (FACHADA)

- COLABORACIONES
  - Los clientes se comunican con el subsistema enviando peticiones al Facade, el cual reenvía a los objetos apropiados.
  - Los clientes que usan la fachada no tienen que acceder directamente a los objetos del subsistema.
- CONSECUENCIAS
  - Oculta a los clientes los componentes del subsistema.
  - Promueve un débil acoplamiento entre el subsistema y sus cliente.
  - No impide que las aplicaciones usen directamente las clases del subsistema si lo necesitan.

EJEMPLO - Clases del Subsistema

SUBSISTEMAS

```
public class Amplifier {
    public void on()                  { System.out.println("Amplifier on"); }
    public void setVolume(int level) { System.out.println("Volume set to " + level); }
}

public class DVDPlayer {
    public void on()                    { System.out.println("DVD Player on"); }
    public void play(String movie)      { System.out.println("Playing: " + movie); }
}

public class Projector {
    public void on()             { System.out.println("Projector on"); }
    public void wideScreenMode() { System.out.println("Projector in widescreen mode"); }
}
```

EJEMPLO - Fachada

INTERFAZ

```java
public interface HomeTheater {
    void watchMovie(String movie);
}
```

```java
public class HomeTheaterFacade implements HomeTheater {

    private Amplifier  amp;
    private DVDPlayer  dvd;
    private Projector  projector;

    public HomeTheaterFacade() {
        this.amp       = new Amplifier();
        this.dvd       = new DVDPlayer();
        this.projector = new Projector();
    }

    // Operación de alto nivel — orquesta los subsistemas
    @Override
    public void watchMovie(String movie) {
        projector.on();
        projector.wideScreenMode();
        amp.on();
        amp.setVolume(10);
        dvd.on();
        dvd.play(movie);
    }
}
```

EJEMPLO - Cliente

CLIENTE

```
public class Main {
    public static void main(String[] args) {

        // Depende de la interfaz, no de la clase concreta
        HomeTheater homeTheater = new HomeTheaterFacade();
        homeTheater.watchMovie("Interstellar");
    }
}
```

Ejercicio - Sistema de Apertura de Cuenta Bancaria Online

Un banco quiere ofrecer a sus clientes la posibilidad de abrir una cuenta bancaria desde su aplicación web. Internamente, el proceso involucra varios subsistemas independientes que ya existen y funcionan por separado:

Subsistema	Responsabilidad
`ServicioVerificacionIdentidad`	Valida DNI y datos personales contra un registro externo
`ServicioRiesgoCredito`	Consulta el historial crediticio del solicitante
`ServicioCreacionCuenta`	Crea la cuenta en el sistema central del banco
`ServicioNotificaciones`	Envía email y SMS de bienvenida al cliente

Ejercicio - Sistema de Apertura de Cuenta Bancaria Online

```java
class ServicioVerificacionIdentidad {
    public boolean verificar(String dni, String nombre) {
        System.out.println("Verificando identidad de " + nombre + "...");
        return true; // simulación
    }
}

class ServicioRiesgoCredito {
    public String evaluarRiesgo(String dni) {
        System.out.println("Evaluando riesgo crediticio...");
        return "BAJO"; // puede ser "BAJO", "MEDIO", "ALTO"
    }
}
```

Ejercicio - Sistema de Apertura de Cuenta Bancaria Online

```java
class ServicioCreacionCuenta {
    public String crearCuenta(String dni, String tipoCuenta) {
        System.out.println("Creando cuenta " + tipoCuenta + "...");
        return "CBU-00123456"; // retorna el CBU asignado
    }
}

class ServicioNotificaciones {
    public void notificar(String email, String mensaje) {
        System.out.println("Enviando notificación a " + email + ": " + mensaje);
    }
}
```

Ejercicio - Sistema de Apertura de Cuenta Bancaria Online

a) Verificar la identidad con ServicioVerificacionIdentidad

b) Si la verificación falla → lanzar excepción o retornar mensaje de error

c) Evaluar el riesgo crediticio con ServicioRiesgoCredito

d) Si el riesgo es "ALTO" → rechazar la solicitud

e) Crear la cuenta con ServicioCreacionCuenta

f) Notificar al cliente con ServicioNotificaciones

g) Retornar el CBU asignado

Ejercicio - Resolución

```
class BancaFachada {

    private ServicioVerificacionIdentidad verificador;
    private ServicioRiesgoCredito evaluadorRiesgo;
    private ServicioCreacionCuenta creadorCuenta;
    private ServicioNotificaciones notificador;

    public BancaFachada() {
        this.verificador     = new ServicioVerificacionIdentidad();
        this.evaluadorRiesgo = new ServicioRiesgoCredito();
        this.creadorCuenta   = new ServicioCreacionCuenta();
        this.notificador     = new ServicioNotificaciones();
    }
```

```java
public String abrirCuenta(String dni, String nombre,
                          String email, String tipoCuenta) {

    // a) Verificar identidad
    boolean identidadOk = verificador.verificar(dni, nombre);
    if (!identidadOk) {
        throw new RuntimeException(
            "Identidad no verificada para DNI: " + dni);
    }

    // b) Evaluar riesgo crediticio
    String riesgo = evaluadorRiesgo.evaluarRiesgo(dni);
    if (riesgo.equals("ALTO")) {
        throw new RuntimeException(
            "Solicitud rechazada por riesgo ALTO.");
    }

    // c) Crear la cuenta
    String cbu = creadorCuenta.crearCuenta(dni, tipoCuenta);

    // d) Notificar al cliente
    notificador.notificar(email,
        "Cuenta creada. CBU: " + cbu);

    return cbu;
}
```

```java
public class Main {
    public static void main(String[] args) {

        // El cliente solo conoce la Fachada
        BancaFachada banco = new BancaFachada();

        try {
            String cbu = banco.abrirCuenta(
                "28.456.789",
                "María García",
                "mgarcia@email.com",
                "Caja de Ahorro"
            );
            System.out.println("Cuenta abierta exitosamente. CBU: " + cbu);
        } catch (RuntimeException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

¿Que podemos responder?

- ¿Qué conocimiento tiene el cliente (Main) sobre los subsistemas antes y después de introducir la Fachada?
- ¿Qué pasa si el banco agrega un nuevo subsistema (ej.: ServicioAntiFraude)? ¿Dónde se hace el cambio?
- ¿La Fachada oculta o elimina los subsistemas? ¿Un cliente avanzado podría seguir usándolos directamente?
- ¿qué principio de diseño promueve el patrón Fachada?

Bridge

- NOMBRE DEL PATRÓN: Bridge
- CLASIFICACIÓN DEL PATRÓN: De Estructura
- INTENCIÓN: desacopla una abstracción de su implementación, de modo que ambas puedan variar independientemente.
- MOTIVACIÓN: algunas veces hay clases con múltiples dimensiones de variaciones (ej. diferentes plataformas, formatos, tipos de dispositivos). Usar herencia hace que sea muy engorroso. El patrón Bridge usa composición en lugar de herencia.

Bridge

- APLICABILIDAD:
  - Cuando se desea evitar el vínculo permanente entre la abstracción y su implementación.
  - Cuando la abstracción y la implementación pueden tener múltiples variaciones.
  - Cuando cambios en la implementación no deberían afectar la abstracción, y viceversa.

Bridge - Ejercicio

Una empresa necesita generar reportes en distintos **formatos** (como **PDF**, **Excel**, o **HTML**) a partir de distintos **tipos de datos**, por ejemplo:

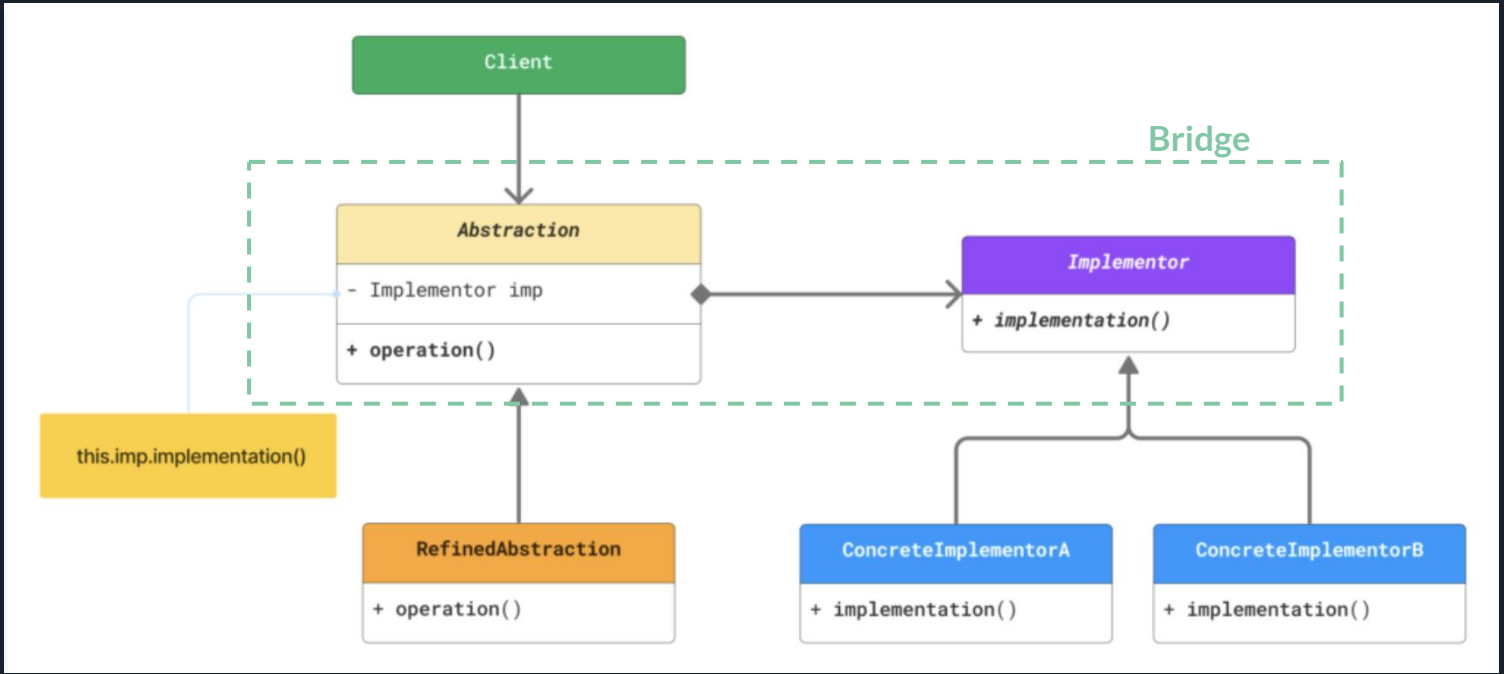
- Reporte de **Ventas**
- Reporte de **Empleados**
- Reporte de **Inventario**

Si mezclamos cada tipo de reporte con cada tipo de formato, podríamos acabar con muchas clases:

- ReporteVentasPDF
- ReporteEmpleadosExcel
- ReporteInventarioHTML

Bridge

- ESTRUCTURA:



Bridge

- PARTICIPANTES:

  Abstraction: Define la interfaz de alto nivel y mantiene una referencia a un objeto de tipo Implementor.

  RefinedAbstraction: Extiende o refina la interfaz definida por Abstraction.

  Implementor: Define la interfaz para las implementaciones de clase.

  ConcreteImplementor: Implementa la interfaz de Implementor.

  -

Bridge - Ejercicio

Con el patrón **Bridge**, el problema se divide en:

- La **abstracción**: qué tipo de reporte es (Ventas, Empleados, etc.)
- De la **implementación**: en qué formato se genera (PDF, Excel, HTML)

Bridge

- Resolvemos en grupo
  - Abstracción
  - Implementación
  - Cliente

Bridge

```
// ── IMPLEMENTACIÓN (jerarquía de formatos) ──────

interface GeneradorFormato {
    void renderizar(String titulo, String datos);
}
```

```
class GeneradorPDF implements GeneradorFormato {
    public void renderizar(String titulo, String datos) {
        System.out.println("[PDF] Generando: " + titulo);
        System.out.println("[PDF] Contenido: " + datos);
    }
}

class GeneradorExcel implements GeneradorFormato {
    public void renderizar(String titulo, String datos) {
        System.out.println("[EXCEL] Generando: " + titulo);
        System.out.println("[EXCEL] Contenido: " + datos);
    }
}

class GeneradorHTML implements GeneradorFormato {
    public void renderizar(String titulo, String datos) {
        System.out.println("[HTML] <h1>" + titulo + "</h1>");
        System.out.println("[HTML] <p>" + datos + "</p>");
    }
}
```

Bridge

```
// ── ABSTRACCIÓN (jerarquía de reportes) ──────────────────────────

abstract class Reporte {

    // El "puente": referencia a la implementación
    protected GeneradorFormato formato;

    public Reporte(GeneradorFormato formato) {
        this.formato = formato;
    }

    // Cada subclase sabe QUÉ datos reunir; el formato los renderiza
    public abstract void generar();
}
```

Bridge

```
class ReporteVentas extends Reporte {
    public ReporteVentas(GeneradorFormato formato) {
        super(formato);
    }
    public void generar() {
        String datos = "Ventas Q1: $120.000 | Q2: $95.000";
        formato.renderizar("Reporte de Ventas", datos);
    }
}
```

Bridge

```
class ReporteEmpleados extends Reporte {
    public ReporteEmpleados(GeneradorFormato formato) {
        super(formato);
    }
    public void generar() {
        String datos = "Total: 48 empleados | Bajas: 2 | Altas: 5";
        formato.renderizar("Reporte de Empleados", datos);
    }
}
```

Bridge

```
class ReporteInventario extends Reporte {
    public ReporteInventario(GeneradorFormato formato) {
        super(formato);
    }
    public void generar() {
        String datos = "Stock: 1.230 unidades | Faltantes: 12";
        formato.renderizar("Reporte de Inventario", datos);
    }
}
```

Bridge

```java
public class Main {
    public static void main(String[] args) {

        // Combinar en tiempo de ejecución, sin new ReporteVentasPDF()
        Reporte r1 = new ReporteVentas(new GeneradorPDF());
        Reporte r2 = new ReporteEmpleados(new GeneradorExcel());
        Reporte r3 = new ReporteInventario(new GeneradorHTML());

        r1.generar();
        r2.generar();
        r3.generar();

        // El mismo reporte, otro formato — sin tocar ninguna clase
        Reporte r4 = new ReporteVentas(new GeneradorHTML());
        r4.generar();
    }
}
```

Mediator

- NOMBRE DEL PATRÓN: Mediator
- CLASIFICACIÓN DEL PATRÓN: De Comportamiento
- INTENCIÓN: el objetivo es **reducir el acoplamiento directo** entre objetos. En lugar de que los objetos se comuniquen entre ellos directamente, lo hacen a través de un **objeto mediador central**, que gestiona la interacción.
- MOTIVACIÓN: En sistemas con muchas clases interconectadas, el cambio en una clase puede afectar a otras, lo que lleva a un sistema frágil y difícil de mantener. La gestión de la comunicación entre múltiples clases o componentes puede volverse compleja y difícil de entender y mantener.

Mediator

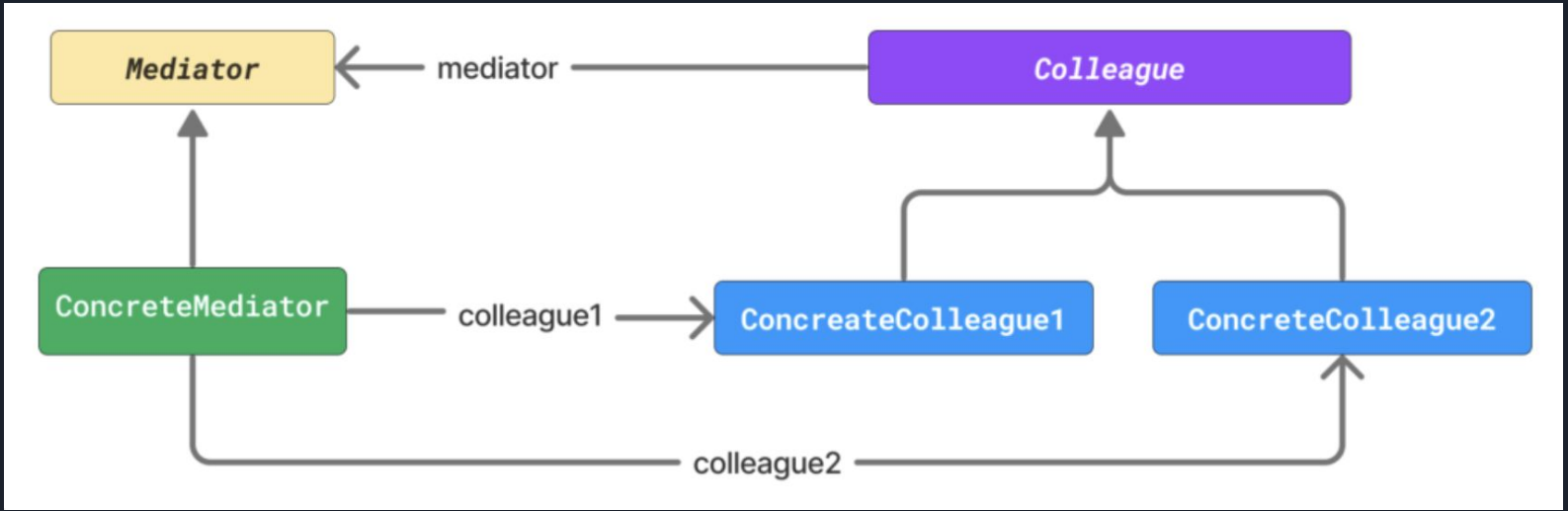
- APLICABILIDAD:
  - Hay muchos objetos interactuando de forma compleja pero bien definida.
  - Se quiere centralizar el control y la lógica de comunicación, de modo que cada objeto no necesite conocer el funcionamiento de los demás.

Mediator - Ejercicio

- Debemos crear un sistema de control de tráfico aéreo para gestionar el espacio aéreo de un aeropuerto concurrido.
- Este sistema necesita manejar múltiples componentes, como torres de control, aviones en vuelo, servicios en tierra y pistas de aterrizaje.
- Permitiendo el intercambio de mensajes entre los componentes.

Mediator

- ESTRUCTURA:



Mediator

- PARTICIPANTES:

  Mediator: Define una interfaz para comunicarse con los objetos Colleague.

  ConcreteMediator: Implementa la interfaz Mediator y coordina la comunicación entre objetos Colleague.

  Colleague: Interfaz común para los objetos que deben comunicarse entre sí a través del Mediator.

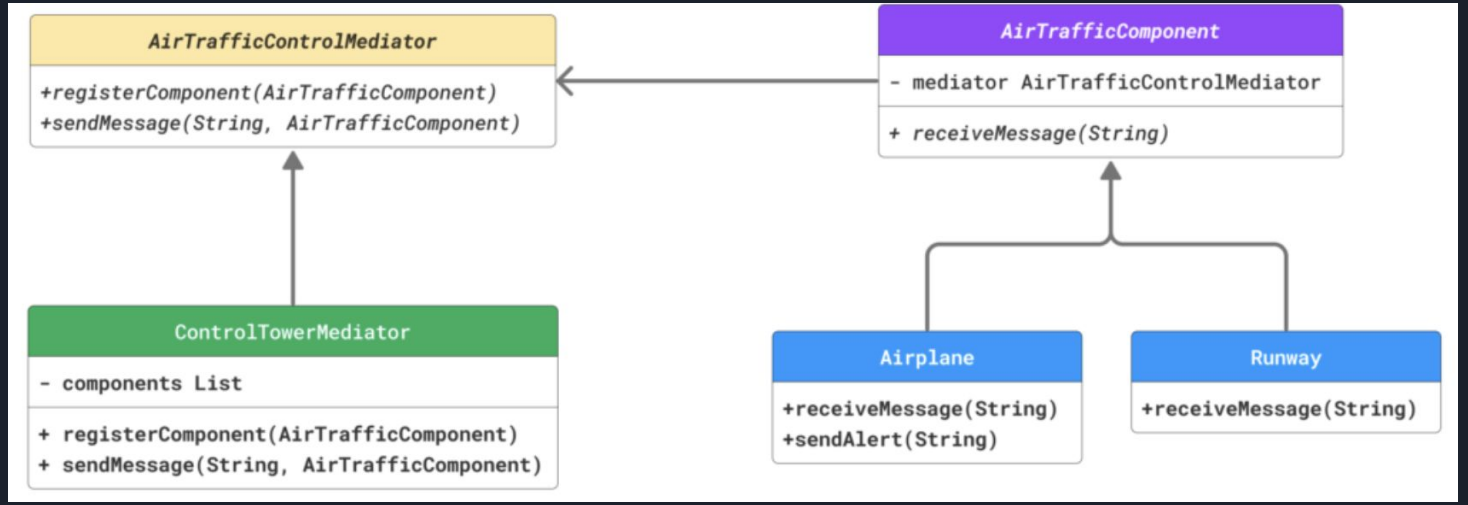
  ConcreteColleague: Clases que se comunican entre sí a través del Mediator.

Mediator - Ejercicio

- Resolvemos en grupo
  - ¿Quien es el mediador?
    - ¿Qué responsabilidades tiene?
  - ¿Que tienen que responder los miembros?
  - ¿Cómo resuelvo?

Mediator

EJEMPLO 1 DE IMPLEMENTACIÓN:



Mediator

- EJEMPLO 1 DE IMPLEMENTACIÓN:

```
interface AirTrafficControlMediator {
    void registerComponent(AirTrafficComponent component);
    void sendMessage(String message, AirTrafficComponent originator);
}
```

Mediator

```
class ControlTowerMediator implements AirTrafficControlMediator {
    private List components = new ArrayList<>();

    @Override
    public void registerComponent(AirTrafficComponent component) {
        components.add(component);
    }

    @Override
    public void sendMessage(String message, AirTrafficComponent originator) {
        for (AirTrafficComponent component : components) {
            // No envia mensaje al originador
            if (component != originator) {
                component.receiveMessage(message);
            }
        }
    }
}
```

Mediator

```
abstract class AirTrafficComponent {
    protected AirTrafficControlMediator mediator;

    public AirTrafficComponent(AirTrafficControlMediator mediator) {
        this.mediator = mediator;
    }

    public abstract void receiveMessage(String message);
}
```

Mediator

```
class Airplane extends AirTrafficComponent {
    public Airplane(AirTrafficControlMediator mediator) {
        super(mediator);
    }

    @Override
    public void receiveMessage(String message) {
        System.out.println("Avión recibió: " + message);
    }

    public void sendAlert(String message) {
        System.out.println("Avión envía alerta: " + message);
        mediator.sendMessage(message, this);
    }
}
```

Mediator

```
class Runway extends AirTrafficComponent {
    public Runway(AirTrafficControlMediator mediator) {
        super(mediator);
    }

    @Override
    public void receiveMessage(String message) {
        System.out.println("Pista recibió: " + message);
    }
}
```

Mediator

```
public class AirportSystem {
    public static void main(String[] args) {
        AirTrafficControlMediator mediator = new ControlTowerMediator();
        Airplane airplane1 = new Airplane(mediator);
        Runway mainRunway = new Runway(mediator);

        mediator.registerComponent(airplane1);
        mediator.registerComponent(mainRunway);

        airplane1.sendAlert("Solicitando aterrizaje.");
    }
}
```

Mediator - Ejercicio

- Tenemos sala de chat
    a. Sala
    b. Usuario
        i. Enviar Mensaje
        ii. Recibir Mensaje

Mediator - Sala de chat

```
// Interfaz del mediador
interface IChatMediator {
    void EnviarMensaje(string mensaje, Usuario remitente, Usuario? receptor = null);
    void RegistrarUsuario(Usuario usuario);
}
```

```
// Clase mediador concreta
class ChatMediator : IChatMediator {
    private List<Usuario> usuarios = new();

    public void RegistrarUsuario(Usuario usuario) {
        if (!usuarios.Contains(usuario))
            usuarios.Add(usuario);
    }

    public void EnviarMensaje(string mensaje, Usuario remitente, Usuario? receptor = null
        if (receptor == null) {
            // Mensaje a todos
            foreach (var usuario in usuarios) {
                if (usuario != remitente)
                    usuario.Recibir($"[Todos] {remitente.Nombre}: {mensaje}");
            }
        } else {
            // Mensaje privado
            receptor.Recibir($"[Privado] {remitente.Nombre}: {mensaje}");
        }
    }
}
```

Mediator

```
// Clase colega (usuario)
class Usuario {
    public string Nombre { get; }
    private IChatMediator chat;

    public Usuario(string nombre, IChatMediator chat) {
        this.Nombre = nombre;
        this.chat = chat;
    }

    public void EnviarMensaje(string mensaje, Usuario? receptor = null) {
        chat.EnviarMensaje(mensaje, this, receptor);
    }

    public void Recibir(string mensaje) {
        Console.WriteLine($"{Nombre} recibe: {mensaje}");
    }
}
```

Mediator

```
// Crear el mediador
var chat = new ChatMediator();

// Crear usuarios
var alice = new Usuario("Alice", chat);
var bob = new Usuario("Bob", chat);
var charlie = new Usuario("Charlie", chat);

// Registrar usuarios
chat.RegistrarUsuario(alice);
chat.RegistrarUsuario(bob);
chat.RegistrarUsuario(charlie);

// Enviar mensaje a todos
alice.EnviarMensaje("¡Hola a todos!");

// Enviar mensaje privado
bob.EnviarMensaje("Hola Alice, ¿cómo estás?", alice);
```

Relación entre patrones

